IY3660 – Applications of Cryptography

Week 8 Lecture
Public-Key Encryption Part 1

Elizabeth Quaglia

Information Security Group

Quiz time!

In today’s lecture

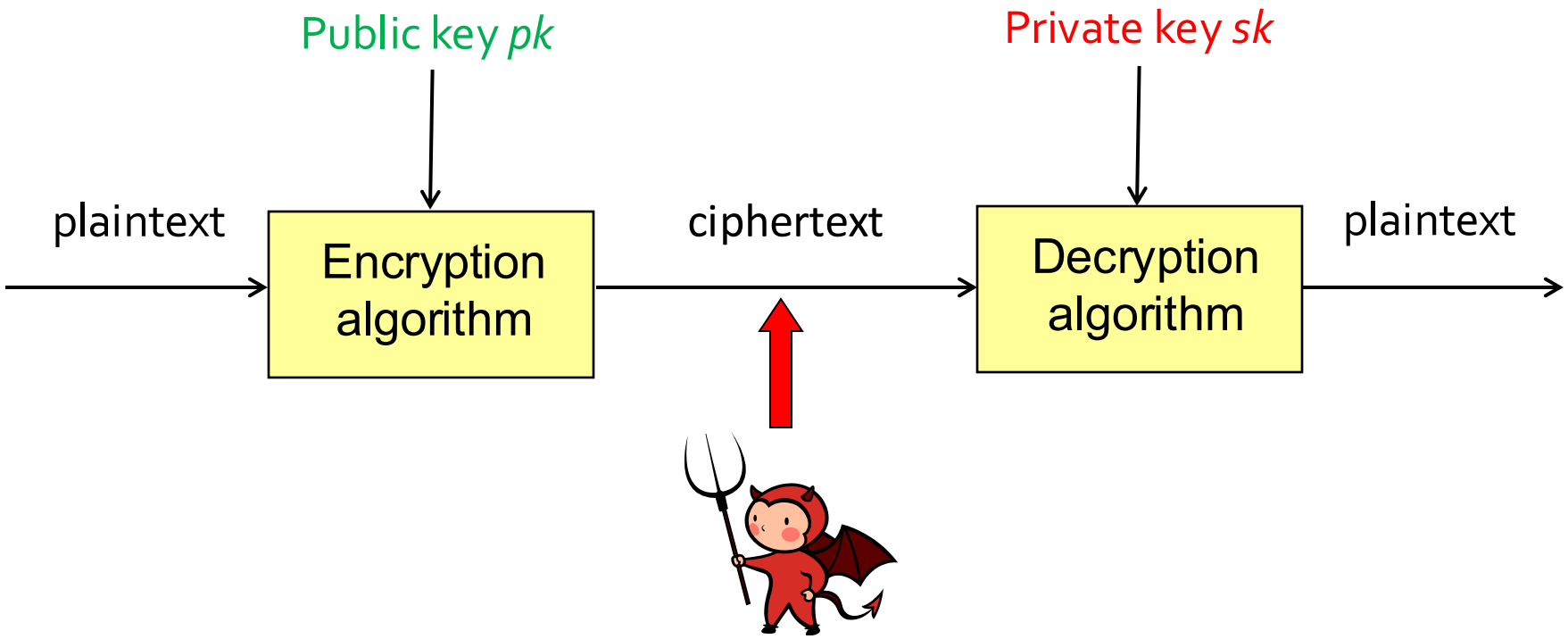
- Introducing Public-Key Encryption
- Basic number theory
- Textbook RSA
- Padding for RSA

Introducing Public-Key Encryption

Introducing Public-Key Encryption

- So far, we have only considered symmetric encryption schemes
  - We know how to protect confidentiality and integrity/authenticity of messages using secure Authenticated Encryption schemes (e.g., AES-GCM)
- However, participants need to somehow share key *K* in order to encrypt/decrypt
- For this goal we will introduce and study a different class of cryptographic schemes
- In **Public-Key Encryption** (PKE), we use different keys for encryption and decryption
  - Bob generates a public/private key pair
  - Alice uses Bob's public key to encrypt to Bob
  - Bob uses his private key to decrypt
- Sometimes called *Asymmetric Encryption*

Secure communications using PKE



- In asymmetric / public key cryptography, the encryption key and decryption key are different
- Keys come in pairs (*sk*,*pk*); public key *pk* is used for encryption, private key *sk* is used for decryption

Formalising PKE

A PKE scheme consists of a triple of algorithms: (**KGen**,**Enc**,**Dec**)

- **KGen**: key generation, generates a key pair (sk,pk)
- **Enc**: encryption, takes as input public key pk, plaintext $m \in \{0, 1\}^*$ and produces output $c \in \{0, 1\}^*$
- **Dec**: decryption, takes as input private key sk, ciphertext $c \in \{0, 1\}^*$ and produces output $m \in \{0, 1\}^*$, or an error message denoted $\perp$

Correctness: for all key pairs (sk,pk) output by **KGen**, and for all plaintexts m, we have $\mathrm{Dec}_{sk}(\mathrm{Enc}_{pk}(m)) = m$

Notes:

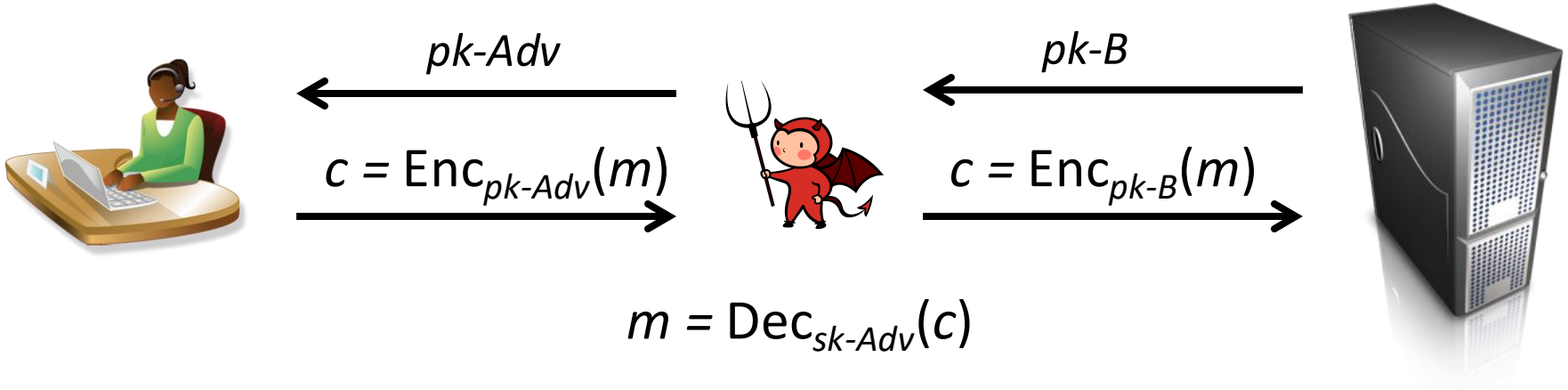
- **Enc** is usually randomised (this is necessary for strong security)
- Terminology: we usually say *private key* for PKE schemes, and *secret key* for symmetric key encryption schemes

What is PKE good for?

- PKE is typically much more expensive than symmetric encryption (in speed, power/energy consumption, key size, etc)
- PKE is often used in applications to transport symmetric keys: this is called hybrid encryption
  - The symmetric keys are then used in a symmetric encryption scheme to encrypt the actual data
  - In this context, PKE is sometimes referred to as being a KEM (Key Encapsulation Mechanism)
- PKE is occasionally used to directly transport short, infrequent messages (e.g., PINs)

Main problem with PKE: distribution of **authentic** public keys

- How does Alice know that a public key is genuinely Bob's?



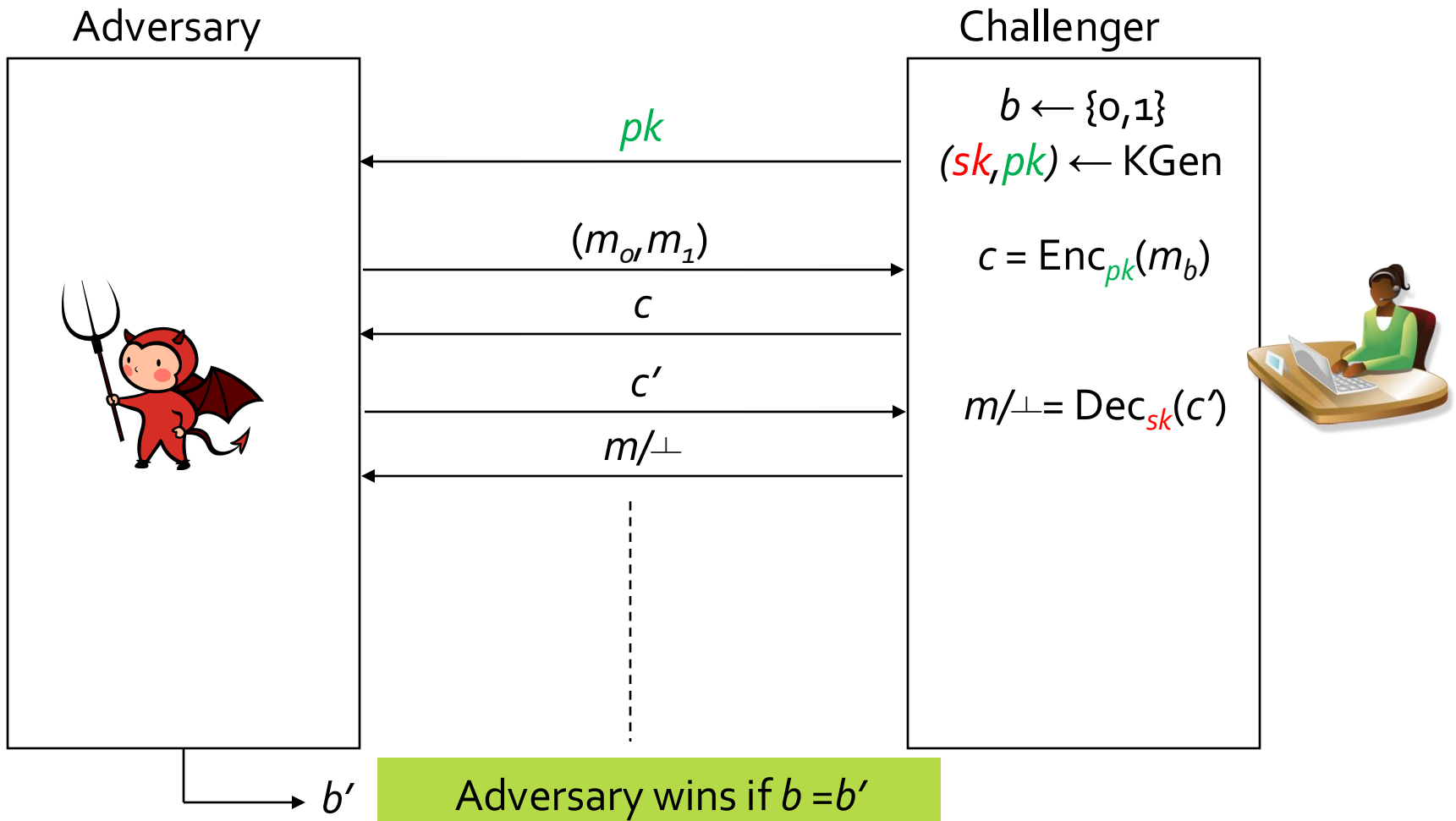
- More on this later, when we discuss key management
- At a high level, PKE translates the **problem of symmetric key distribution** into the **problem of distribution of authentic public keys**

Formalising security for PKE

- Similar to symmetric encryption, it is not enough that the secret key should be hard to guess
- We aim for a high degree of confidentiality of messages, more precisely indistinguishability under chosen ciphertext attacks (IND-CCA)
- This captures the idea that nothing about the plaintext should leak to the adversary, even when the adversary can obtain **decryptions** of arbitrary ciphertexts
- Adversary has access to an *encryption* oracle and a *decryption* oracle for key pair (sk,pk)
- Adversary can submit arbitrary* bit-strings c' to the decryption oracle and receives $\mathrm{Dec}_{sk}(c')$
- Adversary must correctly determine the bit b

* Adversary cannot submit an output from encryption oracle to decryption oracle, otherwise they can win trivially

IND-CCA security game for PKE



Adversary wins if $b = b'$
Advantage = 2. Pr(b=b') - 1.

IND-CCA security for PKE

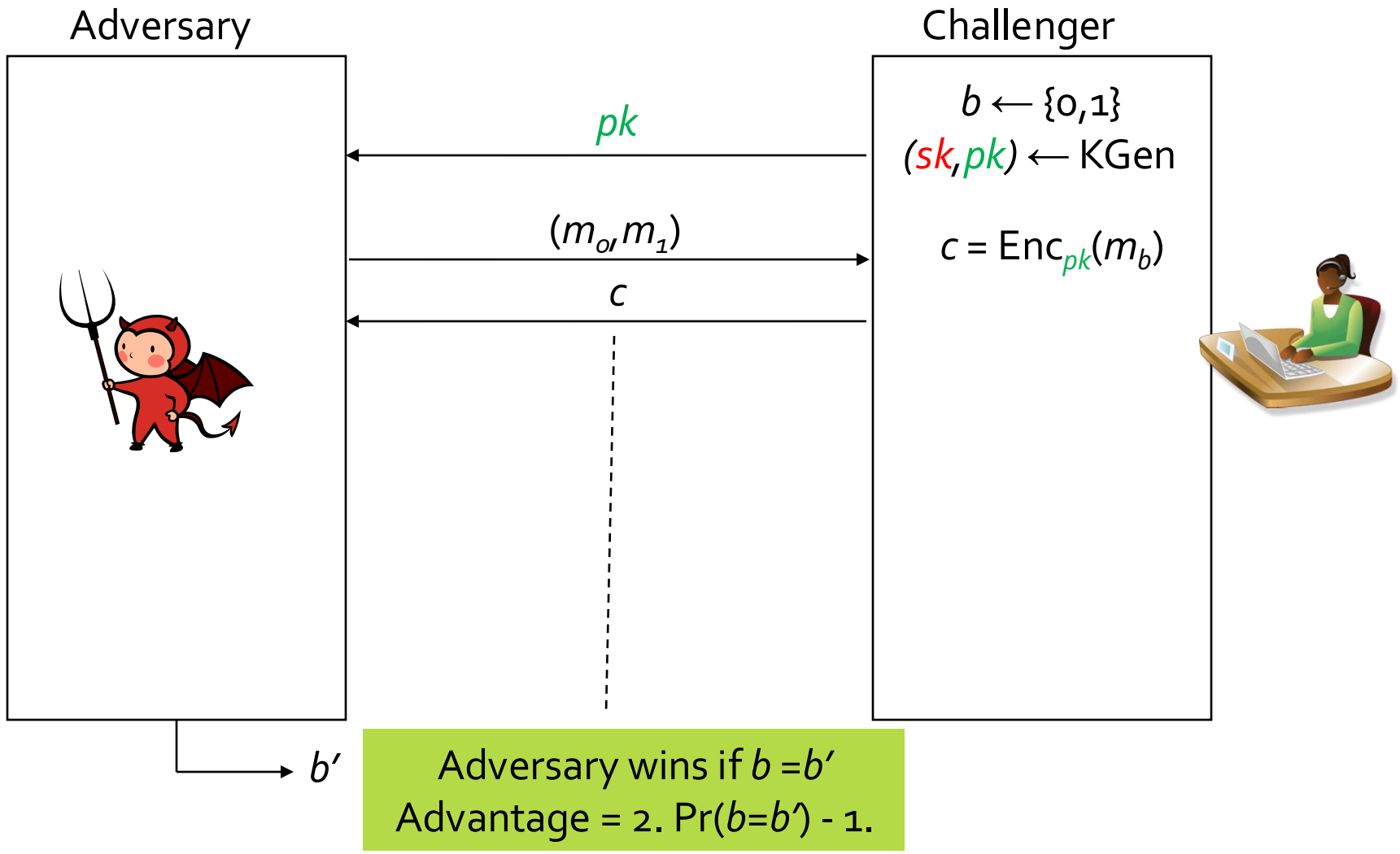
- The adversary's *advantage* in the IND-CCA security game is defined to be:

$$2 \cdot \Pr(b=b') - 1$$

(again, we have "-1" because an adversary can always guess)

- A PKE scheme (**KGen**, **Enc**, **Dec**) is said to be IND-CCA secure if the advantage is "small" for *any* adversary using "reasonable" resources
- Concepts of "small" and "reasonable" can be formalised, but are beyond the scope of this course
- IND-CPA security for PKE: as above, but remove the decryption oracle

IND-CPA security game for PKE



Working with the IND-CCA definition

Q: Suppose an attacker can recover *sk* from *pk* by some means for a PKE scheme. Can the scheme be IND-CCA secure?

A: No. Attacker can just decrypt $c = \mathrm{Enc}_{pk}(m_b)$

Q: Suppose a scheme is IND-CCA secure. Is it necessarily IND-CPA secure?

A. Yes, because removing the decryption oracle only makes the adversary less powerful

Q: Suppose a PKE scheme has a deterministic encryption algorithm. Can it be IND-CPA or IND-CCA secure?

A. No. Make one encryption query on (m_0, m_1) to receive c; then encrypt m_0 and m_1 directly using *pk* and compare ciphertexts

Security of PKE primitives

- **Symmetric** key primitives are typically constructed based on logical operations on bits/bytes
- Most forms of attack try to exploit some non-random property of algorithms
- Goal of the designer is that exhaustive key search is the most efficient form of attack
- On the other hand, **public key** schemes are typically constructed based on algebraic operations
- The private key can be seen as a trapdoor, which is usually based on mathematical problems
- This means that there may be mathematical short-cuts to attack the ciphers
- Thus, even for a good PKE, there are generally better attacks than guessing the secret key
- E.g., for textbook RSA with public key is (e,N)
  - Instead of exhaustively search private key d, we can try to factorise N into its factors p and q
  - Then we solve $ed = 1 \bmod (p-1)(q-1)$ to recover d
  - Hence breaking textbook RSA is no harder than factoring a k-bit integer that is a product of two $k/2$-bit primes

Basic number theory

Remainders and factors

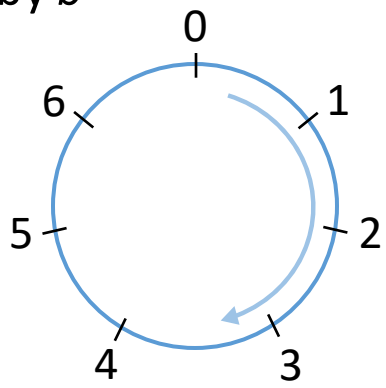
- We use the notation **N** to denote the non-negative integers:

$$\mathbf{N} = \{0,1,2,\ldots\}$$

- We write a mod b for the "remainder of a on division by b", i.e., the integer r in $\{0,\ldots,b-1\}$ such that $a = q \cdot b + r$ for some q
- For example,
  - 9 mod 7 = 2, because $9 = 1 \cdot 7 + 2$
  - 21 mod 7 = 0, because $21 = 3 \cdot 7 + 0$
- We say "a is a factor of b" if a divides b with remainder 0
- For example,
  - 3 is a factor of 6, but
  - 4 is not a factor of 6

Modular arithmetic

- Modular arithmetic refers to arithmetic done with the numbers in $\{0,\ldots,b-1\}$ for some b, where we always reduce our results to remainders on division by b
- Sometimes called: clock arithmetic
- For example, with $b = 7$, we have:
  - $6 + 4 = 10 = 3 \bmod 7$
  - $6 \cdot 4 = 24 = 7 \cdot 3 + 3 = 3 \bmod 7$
  - $6^4 = 6 \cdot 6 \cdot 6 \cdot 6 = (6^2)^2 = 36^2 = 1^2 = 1 \bmod 7$



- Important principle: we never need to work with numbers much bigger than b if we are doing arithmetic mod b; we can just reduce mod b as we go along
  - For example, to compute $5^9 \bmod 7$, we would compute $5^2 = 25 = 4 \bmod 7$, then $4^2 = 16 = 2 \bmod 7$, then $2^2 = 4 \bmod 7$. Thus $5^8 = 4 \bmod 7$, and then $5^9 = 4 \cdot 5 = 20 = 6 \bmod 7$. Thus $5^9 = 6 \bmod 7$.
- Note that if b is composite, then we can multiply two non-zero numbers mod b and get zero!
  - For example, with $b = 12$ we have $6 \cdot 4 = 24 = 2 \cdot 12 + 0 = 0 \bmod 12$
- This is different from normal arithmetic….

Prime numbers

- A natural number (i.e., in **N**) greater than 1 with only itself and 1 as factors is called a prime number
- For example, 13 is prime, but $21 = 3 \cdot 7$ is composite
- A few facts:
  - The number 1 is **not** a prime
  - Every positive integer can be written as a product of primes, and this factorization is unique (up to reordering the primes)
    - This is known as the Fundamental Theorem of Arithmetic
    - E.g., $84 = 2 \cdot 2 \cdot 3 \cdot 7$ is the unique factorization of 84 as product of primes
  - There are infinitely many primes
  - The number of primes up to a positive integer N is around N/ln(N)

Primes and gcds

- Given a, b two positive integers, the greatest common divisor (gcd) of a and b is the largest number that is a factor of both numbers

- If you write a and b using their prime factorization, the gcd will be the product of the common (prime) factors
  - E.g., $4 = 2 \cdot 2$ and $6 = 3 \cdot 2$. Then $\gcd(4,6) = 2$
  - E.g., $84 = 2 \cdot 2 \cdot 3 \cdot 7$ and $66 = 2 \cdot 3 \cdot 11$. Then $\gcd(66,86) = 2 \cdot 3 = 6$

- Two numbers are said to be relatively prime if their gcd is 1
  - That is, they have no common prime factors

Modular inverses

- Over the rational numbers we define the (multiplicative) inverse of a, denoted a^{-1} as the number such that $a \cdot a^{-1} = 1$
- Modulo some number p, we can also define (modular) inverses
  - For example, $5 \cdot 5 \bmod 6 = 25 \bmod 6 = 1 \bmod 6$
  - Hence $5^{-1} = 5 \bmod 6$
- All numbers $0 < a < p$ such that $\gcd(a,p) = 1$ have an inverse mod p
- If $0 < a < p$ is such that $\gcd(a,p) > 1$, then a will not have an inverse mod p
- Now if p is a prime, then $\gcd(a,p) = 1$ for all $0 < a < p$ (why?), and so every number a in $\{1,\ldots,p-1\}$ will have an inverse mod p
- The numbers modulo a prime p form a field, which means that addition, multiplication and division are defined and work for all inputs "as expected"

Example: arithmetic modulo 5

- As an example, let's make the addition and multiplication tables mod *5*

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

·	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

- Notice the pattern along rows and columns of addition table
- Notice how every element has a multiplicative inverse except 0
- 1 and 4 are self-inverse
- As an exercise, make the same tables for modulus 6

Euclidean algorithm for computing gcd

- Given $a, b \in \mathbf{N}$ we want to compute $\gcd(a,b)$
- Euclid's algorithm is based on recursive application of the formulae:

$$\gcd(a,b) = \gcd(b, a \bmod b) \qquad (1)$$

$$\gcd(a,0) = a \qquad (2)$$

Notice that $0 \leq a \bmod b < b$, so the numbers decrease in size at every step

- Example:

$$\begin{aligned} &\gcd(21, 14) \\ &= \gcd(14, 21 \bmod 14) && \text{(using (1))} \\ &= \gcd(14, 7) \\ &= \gcd(7, 14 \bmod 7) && \text{(using (1))} \\ &= \gcd(7,0) \\ &= 7. && \text{(using (2))} \end{aligned}$$

The Extended Euclidean algorithm

- The Extended Euclidean algorithm outputs integers r, s, t where $r = \gcd(a,b)$ and $s \cdot a + t \cdot b = r$
- It gives an expression for $\gcd(a,b)$ as a linear combination of a and b over the integers
- It proceeds like the Euclidean algorithm but keeps track of additional information
- It computes a series of coefficients r_i , s_i , t_i such that:

$$r_i = s_i \cdot a + t_i \cdot b$$

- In each step we compute

$$r_{i+1} = r_{i-1} - q_i \cdot r_i \qquad \text{where} \quad 0 \le r_{i+1} < |r_i|$$
$$s_{i+1} = s_{i-1} - q_i \cdot s_i$$
$$t_{i+1} = t_{i-1} - q_i \cdot t_i$$

- We initialise the algorithm with $r_0 = a$, $r_1 = b$ and $s_0 = 1$, $s_1 = 0$, $t_0 = 0$, $t_1 = 1$
- Eventually $r_k = \gcd(a,b)$ and $\gcd(a,b) = s_k \cdot a + t_k \cdot b$

Extended Euclidean algorithm: example

i	r_i	q_i	s_i	t_i	$r_i = s_i \cdot a + t_i \cdot b$
0	a = 14	—	1	0	$1 \cdot 14 + 0 \cdot 21 = 14$
1	b = 21	—	0	1	$0 \cdot 14 + 1 \cdot 21 = 21$
2	14	0	1	0	$1 \cdot 14 + 0 \cdot 21 = 14$
3	7	1	-1	1	$\boxed{-1 \cdot 14 + 1 \cdot 21 = 7}$

Recall, we initialise with $r_0 = a$, $r_1 = b$ and $s_0 = 1$, $s_1 = 0$, $t_0 = 0$, $t_1 = 1$, and in each step, we compute

$$r_{i+1} = r_{i-1} - q_i \cdot r_i \qquad \text{where} \qquad 0 \le r_{i+1} < |r_i|$$

$$s_{i+1} = s_{i-1} - q_i \cdot s_i$$

$$t_{i+1} = t_{i-1} - q_i \cdot t_i$$

Eventually $\boxed{\mathbf{r_k = \gcd(a,b) \text{ and } \gcd(a,b) = s_k \cdot a + t_k \cdot b}}$

Using the Extended Euclidean algorithm

- The numbers s and t are often useful for applications
- For example, running the extended Euclid's algorithm on a with $0 < a < p$ and a prime p gives s and t such that:

$$a \cdot s + p \cdot t = 1$$

- Reducing this equation mod p, we get:

$$a \cdot s = 1 \bmod p$$

- This allows us to find the inverse of a mod p

Textbook RSA

A concrete example of PKE: textbook RSA

KGen: generates:

- random primes p, q of some bit-size $k/2$, with $N = pq$ (so bit-size of N is k)
- integers d, e such that $de = 1 \bmod (p-1)(q-1)$

  Outputs key pair (sk,pk) where $sk = d$, $pk = (e,N)$

Enc: inputs public key $pk = (e,N)$, plaintext $m \in [1,N-1]$; outputs $c = m^e \bmod N$

Dec: inputs private key $sk = d$, ciphertext c; outputs $m = c^d \bmod N$

Correctness: follows from property that if $de = 1 \bmod (p-1)(q-1)$ then $m^{de} = m \bmod N$ for all $m \in [1,N-1]$ (this demands a mathematical proof)

- Implementation issues: how do we generate random primes of a given bit-size? How do we generate d and e?
- Enc is not randomised here, which already hints at trouble
- **This scheme is not secure! It must not be used as is in practice**

RSA KeyGen in more detail

- Parameterised by k, the bit length of the modulus
  - E.g. RSA-2048 works over a modulus N with k=2048 bits
- Generate two primes p and q with $k/2$ bits
  - Pick a randomly generated ($k/2$)-bit string, set the least/most significant bits to 1 (to ensure it indeed has $k/2$ bits, and it is odd), then run through a primality test
    - If prime, return; otherwise repeat until you have p and q primes
  - There are **many** primes of length ($k/2$) bits, so the chances of two RSA instances using the same primes is negligible
    - See *Further Reading & Research* for example when it goes wrong
- Select e (between 3 and $(p-1)(q-1)$) such that $\gcd(e, (p-1)(q-1)) = 1$
  - In practice, many implementations use e = 65537 or 3 (both are primes)
- Then compute d such that $d \cdot e = 1 \bmod (p-1)(q-1)$ using the Extended Euclidean Algorithm
    - We expect d to have as many bits as N, i.e., k bits and random-looking
- Set $N = pq$ and output key pair (sk, pk) where $sk = d$, $pk = (e, N)$

RSA Enc and Dec in more detail

Enc:

- Inputs public key $pk = (e,N)$, plaintext $m \in [1,N\text{-}1]$
- Output the ciphertext $c = m^e \bmod N$
- The plaintext must be considered as an integer, so may need to define some encoding scheme
  - In practice plaintext might be, e.g., a 128-bit string
- Encryption operation consists of algebraic operations on very large numbers (compare to AES operations!), so efficiency is an issue

Dec:

- Inputs private key $sk = d$, ciphertext c
- Output $m = c^d \bmod N$
- Usually, the private exponent d has no obvious structure, unlike the public exponent e
- Decryption is typically much more (computationally) expensive

Security of RSA

- RSA encryption is "e^{th}-power exponentiation" mod N
  - Easy to compute
- RSA decryption is "e^{th}-root operation" mod N (i.e., $m = \sqrt[e]{c}$ mod N)
  - Hard to compute without knowing the trapdoor d , which allows the eth-root operation via the dth-power exponentiation
- If we know primes p and q (and e), then we can compute d
  - Recall that $de = 1$ mod $(p-1)(q-1)$, and we can use the extended Euclidean algorithm to compute d
- Conversely, one can show that if we know d, e and N, then we can also compute p and q
- **RSA key recovery** and **RSA modulus factorization** are equivalent

Security of RSA

- We saw that **RSA key recovery** == **RSA modulus factorization**
- Does **RSA message recovery** == **RSA modulus factorization**?
  - Probably not
  - The direction (⇐) is certainly true
  - The other direction (⇒) is believed not to be true
  - It is believed that message recovery is likely to be easier than factorising N
- Assuming robust key generation and enc/dec implementation, modulus factorization represents the best-known attack against RSA
  - More efficient than guessing the private key d
- The complexity of factorising N is the main factor we use when selecting RSA parameters
  - We need to choose large enough N so that recovering p and q is hard

Keysize requirements

- The Integer Factorisation Problem (IFP) has been studied for thousands of years
- Study intensified since the 1970s because of its importance in cryptography
- We often say that the security of RSA relies on the hardness of the Integer Factorisation Problem
- The current best algorithm for solving IFP on a classical computer is called the Number Field Sieve (NFS)
  - Its analysis helps determine secure parameters
- The NFS algorithm was invented in the early 1990s, and there has been no significant improvement since then

Keysize requirements

- A **512-bit RSA modulus** can be factored using NFS on Amazon EC2 in about 4 hours at a cost of USD75
  - *See https://eprint.iacr.org/2015/1000*
- Biggest modulus publicly factored is a **829-bit** (250 decimal digit) RSA modulus (in Feb 2020)
  - Performed with the Number Field Sieve algorithm, with total computation time of roughly 2700 core-years, using Intel Xeon Gold 6130 CPUs as a reference (2.1GHz)
  - See https://eprint.iacr.org/2020/697.pdf
- It is conjectured that a **1024-bit modulus** would require about 2^{80} machine operations
  - This would represent 80-bit security and is well beyond the capabilities of academic groups
  - May be within reach for a national security agency with a budget of USD 100Ms
- To achieve 128-bit security, the modulus N must be between **2048** and **3072** bits
  - Estimates vary – BSI vs NIST vs ECRYPT

Keysizes vs estimated security: https://www.keylength.com

Date	Security Strength	Symmetric Algorithms	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash (A)	Hash (B)
Legacy (1)	80	2TDEA	1024	160	1024	160	SHA-1 (2)	
2019 - 2030	112	(3TDEA) (3) AES-128	2048	224	2048	224	SHA-224 SHA-512/224 SHA3-224	
2019 - 2030 & beyond	128	AES-128	3072	256	3072	256	SHA-256 SHA-512/256 SHA3-256	SHA-1 KMAC128
2019 - 2030 & beyond	192	AES-192	7680	384	7680	384	SHA-384 SHA3-384	SHA-224 SHA-512/224 SHA3-224
2019 - 2030 & beyond	256	AES-256	15360	512	15360	512	SHA-512 SHA3-512	SHA-256 SHA-512/256 SHA-384 SHA-512 SHA3-256 SHA3-384 SHA3-512 KMAC256

- Most websites now use RSA public keys of 2048 bits, a few use 3072

RSA: performance

- Complexity of modular exponentiation is cubic in the size of the parameters
  - We assume all operands have the same length *k*-bit
- Improvements:
  - by choosing *e* with low weight (for example, 3 or 65537 = $2^{16}+1$) we can reduce the complexity of encryption to quadratic
  - by using the Chinese Remainder Theorem (CRT) in the operations, we can reduce the complexity of decryption to cubic in *k/2* (rather than *k*)
- Performance of RSA is a bottleneck for the increase of parameters
  - Doubling the size of modulus means decryption is about 8 times as slow

RSA security: malleability

- We know that textbook RSA is not IND-CPA secure
  - We are going to need some kind of randomisation
- But this is not all: textbook RSA also has a troubling **malleability** property
  - Suppose $c = m^e \bmod N$
  - Choose s at arbitrarily from $[1, N-1]$
  - Now compute s^e and then $s^e \cdot c \bmod N$:

$$s^e \cdot c = s^e \cdot m^e = (s \cdot m)^e \bmod N$$

  - We see that $s^e \cdot c \bmod N$ is an encryption of $s \cdot m \bmod N$
  - This gives an active attacker the ability to modify textbook RSA plaintexts in a controlled fashion, by carefully modifying ciphertexts
  - This leads to various attacks against practical systems, as we shall see

RSA security: attacks against small public exponent e

- As we saw, using small exponents, such as $e = 3$, can speed up encryption
- Now suppose Alice has public key ($e=3$, N) and suppose Bob is only sending a very small message $m < N^{1/3}$ to Alice.
  - Then $c = m^3$ over the integers, i.e., no modular reduction has taken place
  - Example:

    N has 1024 bits, m is an AES key with 128 bits, $e = 3$.

    Then m^e has only 3 x 128 = 384 bits, less than 1024 bits.
- If Eve knows that m will be very small, then she can recover the message m from the ciphertext c simply by taking cube roots over the integers
  - Easy to compute using Newton's method
- To mitigate such attacks, **we need some kind of message padding**
- **Note:** Small d is also insecure even up to $d \approx N^{1/4}$
  - Weiner's attack; see the report "Cryptanalysis of RSA: A Survey" in *Further Reading & Research*

Summary of textbook RSA security

- There are several security issues with textbook RSA
  1. There is **no randomness**
     - Scheme is deterministic so not IND-CPA
  2. There is a **short message** attack
  3. Ciphertexts have a **malleability** property

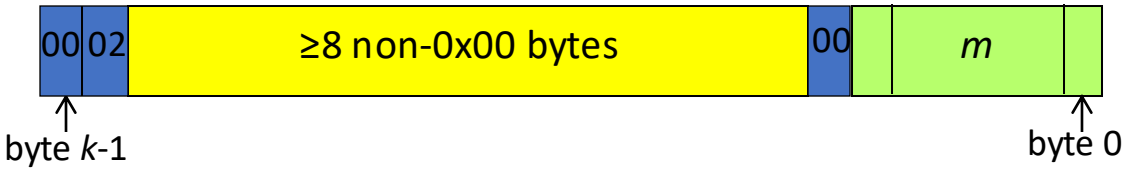
- Solution: Padding!

Padding for RSA

Padding for RSA

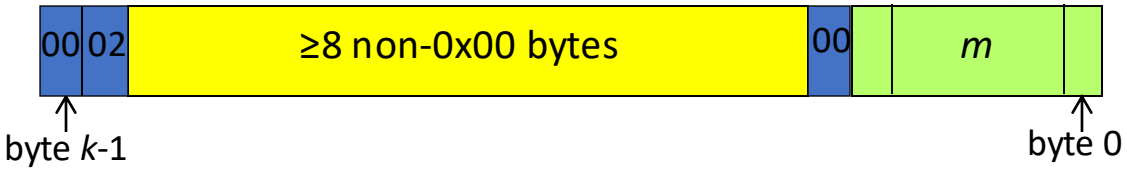
- In response to attacks special padding schemes for RSA were introduced
- The three goals of padding schemes are to:
  1. Introduce **randomness** into the message
  2. **Expand short messages** to full size
  3. Destroy algebraic relationships among messages to **remove the malleability** property
- The ultimate aim is to achieve IND-CCA security for RSA encryption
- This has proved tricky to do
  - One of the most widely-deployed RSA padding schemes, PKCS#1 v1.5, does not attain it
- The padding scheme RSA-OAEP enables us to achieve IND-CCA security
  - This and other padding schemes achieving IND-CCA are not as widely deployed

PKCS#1 v1.5



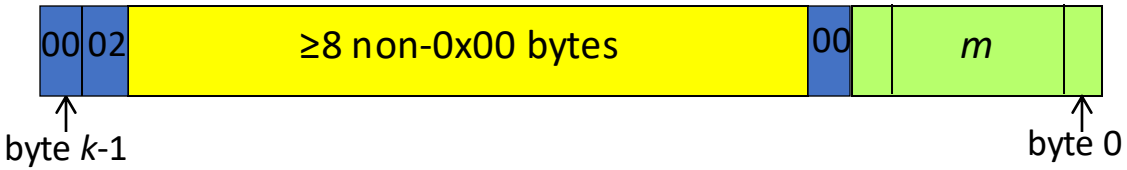
- Byte-oriented encoding scheme for RSA
- Assume *N* has *k* bytes
- Message *m* placed in least significant bytes
- Force first two most significant bytes to 0x00 0x02
  - This ensures resulting encoded message lies between 0 and *N*
- Follow with at least 8 non-0x00 random bytes, then a 0x00 byte
- Maximum message size is *k* - 11 bytes

$\mathbf{Dec}_d(c)$ **with** PKCS#1 v1.5 **padding**



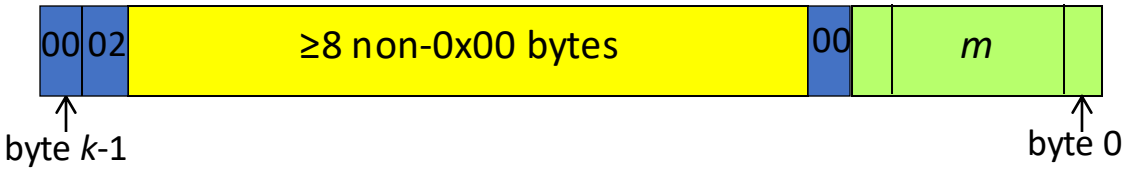
- Compute $m' = c^d \bmod N$
- Check that m' begins with 0x00 0x02, reject if not
- Check that m' then has at least 8 non-0x00 bytes, reject if not
- Check that m' then has a 0x00 byte
  - Reject if least significant byte is reached
- Return as m all the bytes to the right of the 0x00 bytes
- **Is this secure?**

Bleichenbacher's attack on PKCS#1 v1.5



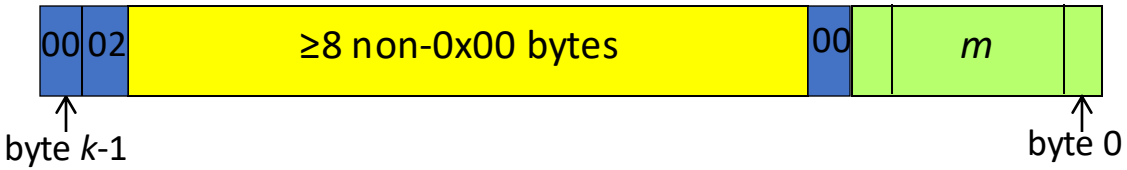
- A random string of bytes has probability roughly 2^{-16} of having a valid padding format
- This is the probability that:
  - The first two bytes are 0x00 0x02; and
  - There are at least 8 non-0x00 bytes; and
  - There is at least one 0x00 byte before the end is reached
- Suppose now that the attacker has an **oracle** telling it whether the decryption algorithm succeeds or fails on an input *c'*
- As with padding oracles for CBC mode, having such an oracle is common in implementations

Bleichenbacher's attack on PKCS#1 v1.5



- Attacker wants to decrypt $c = (00\ 02 || r || 00 || m)^e \bmod N$
- Let's write $p = (00\ 02 || r || 00 || m)$ to denote the encoded message
- Attacker asks for decryption of $s^e \cdot c = (s \cdot p)^e \bmod N$, for some choice of s
  - Most choices of s will lead to a decryption failure
  - But with probability roughly 2^{-16}, decryption succeeds, and the adversary learns that $s \cdot p \bmod N$ begins with bytes 0x00 0x02
- This gives an inequality over the integers:
  - $2 \cdot 256^{k-2} \leq s \cdot p \bmod N \leq 3 \cdot 256^{k-2}$
- By gathering many such inequalities for different and carefully chosen values of s, the attacker can recover p, and hence m
- Details of how that is done are quite complex and omitted here

Bleichenbacher's attack on PKCS#1 v1.5



- The attack requires many decryption attempts, around 2^{20} in its first version (1998)
  - It is known as the million-message attack
- The attack has been improved considerably since
- Many real-world systems, including SSL/TLS, have been found to be vulnerable to it
- RSA-PKCS#1 v1.5 should be avoided in general, but is still widely used
- The attack does not use a full decryption oracle
  - Only a "encoding validity" oracle
  - Similar to a padding oracle we saw for CBC
- The attack can be easily modified to show that RSA-PKCS#1 v1.5 is not IND-CCA secure

RSA-OAEP

- OAEP is an alternative encoding scheme for RSA due to Bellare and Rogaway
- OAEP stands for "optimal asymmetric encryption padding"
- A version of it is widely standardised and called PKCS#1 v2.0

<u>Setup of OAEP</u> using λ-bit RSA moduli (e.g., $\lambda = 2048$)

- Let λ_0 and λ_1 be chosen so that no adversary can perform 2^L operations for $L = \lambda_0$ or λ_1
    - E.g., $\lambda_0 = \lambda_1 = 128$
- Set $n = \lambda - \lambda_0 - \lambda_1$
- Messages in RSA-OAEP are assumed to be n-bit strings
- Let G be a hash function mapping λ_0 bit strings to $n + \lambda_1$ bit strings
- Let H be a hash function from $n + \lambda_1$ bit strings to λ_0 bit strings
- These hash functions should be collision resistant

RSA-OAEP

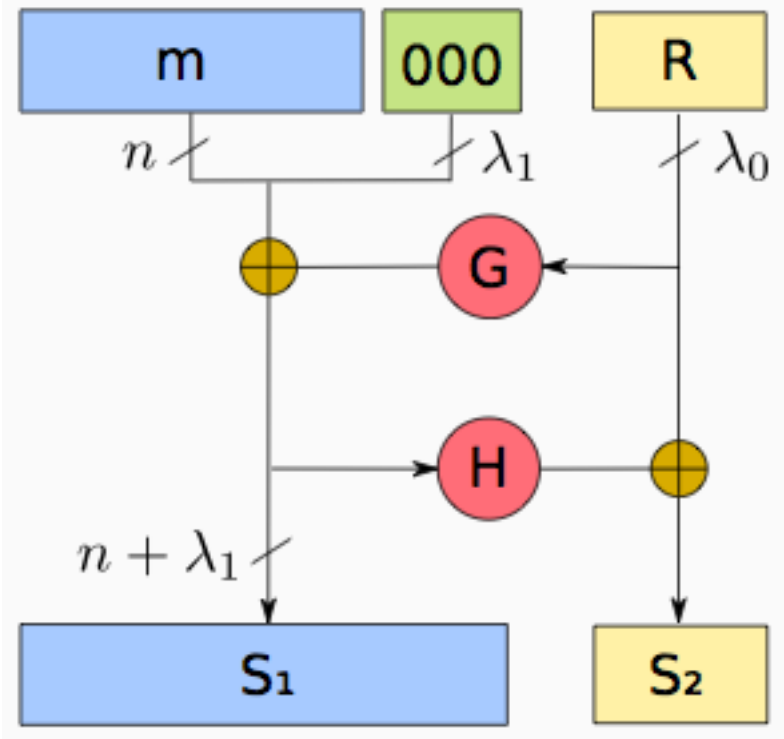
<u>Encryption</u>

- Let m be an n-bit string
- Choose a random λ_0-bit string R
- Set $S_1 = (m \,||\, 0^{\lambda_1}) \oplus G(R)$
- Set $S_2 = R \oplus H(S_1)$
- Form the bit-string $S = S_1 \,||\, S_2$
- Interpret S as a λ-bit integer and compute

$$c = S^e \bmod N$$

<u>Decryption</u>

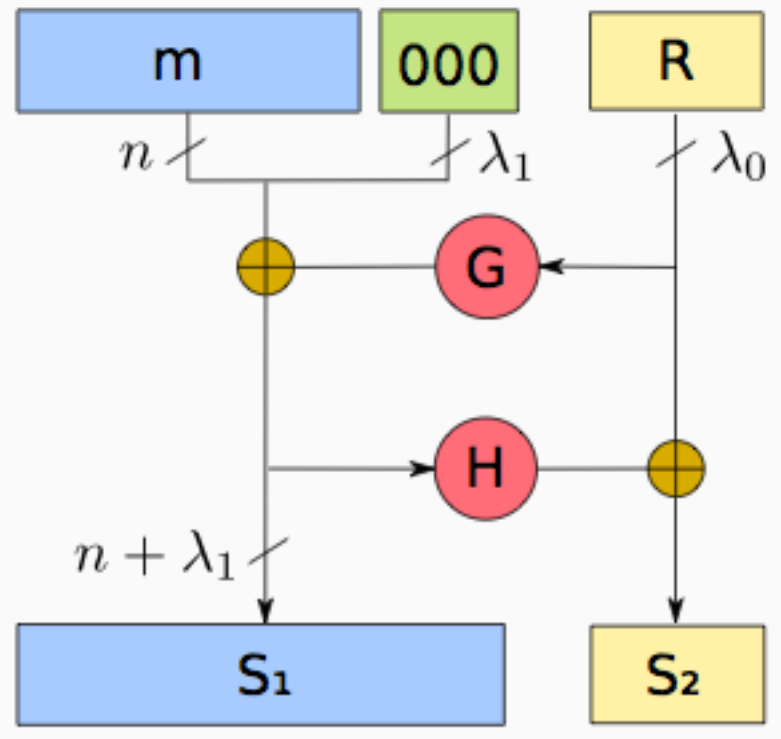
- Computes $R = S_2 \oplus H(S_1)$
- Check that the string $S_1 \oplus G(R)$ has λ_1 0-bits in the right position to recover m
- Decryption fails if not



Rationale for RSA-OAEP

Use RSA-OAEP!

- RSA messages are randomised and have full length
- Since S_1 and S_2 are both outputs of hash functions, the bit-string S looks random and algebraic relationships among the message components are not preserved
- If you are given a decryption oracle and you send it a random value mod N, then the decryption process will fail with high probability, so the decryption oracle essentially becomes useless



- RSA-OEAP can be proven to be IND-CCA secure, though with strong assumptions on G and H, and under a strong number theoretic assumption (much stronger than assuming that factoring is hard)

Homework

- Read Sections 5.1 and 5.2 in "*Everyday Cryptography*"
- Read Chapter 9 and Chapter in "*Serious Cryptography*"
- Review the basic number theory concepts presented in this lecture
- Check the Further Reading & Research section for
  - historic notes about the invention of public-key cryptography
  - more on basic number theory for cryptography